

Práctica Backend Avanzado – BookShop API

Objetivo

Desarrollar una API REST para una plataforma de compraventa de libros de segunda mano.

Los usuarios podrán registrarse, publicar libros, consultar el catálogo disponible y comprar libros publicados por otros usuarios.

La plataforma también ejecutará procesos automáticos de notificación mediante correo electrónico para ayudar a los vendedores a gestionar sus publicaciones.

Requisitos técnicos

La solución deberá desarrollarse utilizando el mismo stack tecnológico que en clase.

Modelo de datos

User

id: number
email: string
password: string
createdAt: Date

Reglas

- El email debe ser único.
- La contraseña debe almacenarse hasheada.
- No se puede almacenar ninguna contraseña en texto plano.
- Un usuario puede publicar múltiples libros.

Book

id: number
title: string
description: string
price: number
author: string
status: "PUBLISHED" | "SOLD"
ownerId: string
soldAt: Date | null
createdAt: Date

Reglas

- Cada libro pertenece a un único usuario.
 - Los libros se crean siempre con estado **PUBLISHED**.
 - Los libros vendidos (**SOLD**) no deben aparecer en los endpoints públicos.
 - Solo el propietario puede editar un libro.
 - Solo el propietario puede eliminar un libro.
 - Un usuario no puede comprar sus propios libros.
 - Un libro vendido no puede volver a comprarse.
 - Mientras el libro esté publicado:
 - `status = PUBLISHED`
 - `soldAt = null`
 - Cuando se complete una compra:
 - `status = SOLD`
 - `soldAt` contendrá la fecha y hora de la venta.
-

Autenticación

La autenticación deberá implementarse mediante JWT.

Todos los endpoints privados deberán protegerse mediante middleware de autenticación.

Registro

POST /authentication/signup

Permite crear una nueva cuenta de usuario.

Inicio de sesión

POST /authentication/signin

Permite autenticar un usuario y devolver un JWT válido.

Gestión de libros

Todos los endpoints de esta sección son privados y requieren autenticación.

Crear libro

POST /books

Campos permitidos

```
{  
  "title": "Clean Code",  
  "description": "Libro en perfecto estado",  
  "price": 20,  
  "author": "Robert C. Martin"  
}
```

Comportamiento esperado

- El libro se asocia al usuario autenticado.
 - El estado inicial será **PUBLISHED**.
 - **soldAt** se inicializará a **null**.
-

Editar libro

PUT /books/:id

Campos editables

- title
- description
- price
- author

Restricciones

- Solo el propietario puede editar el libro.
 - No se puede modificar ninguna propiedad más que las editables.
-

Eliminar libro

DELETE /books/:id

Restricciones

- Solo el propietario puede eliminar el libro.
 - Solo se pueden eliminar libros publicados, los vendidos no.
-

Comprar libro

POST /books/:id/buy

Validaciones

- El libro debe existir.
- El libro debe encontrarse en estado **PUBLISHED**.
- El comprador debe estar autenticado.
- El comprador no puede ser el propietario del libro.

Resultado esperado

- El libro pasa a estado **SOLD**.
 - Se establece el valor de **soldAt**.
 - Se envía un email al vendedor notificando la venta.
-

Consultar mis libros

GET /me/books

Devuelve todos los libros del usuario autenticado, independientemente de su estado.

Catálogo público

Consultar libros

GET /books

Endpoint público.

Requisitos

- Solo devolver libros con estado **PUBLISHED**.
- Implementar búsqueda parcial por:
 - título
 - autor
- La paginación es obligatoria.

Parámetros

page - Número de página

limit - Número de elementos por página

search - Texto de búsqueda

Ejemplo

GET /books?page=1&limit=10&search=rowling

Comportamiento esperado

La búsqueda debe devolver coincidencias parciales tanto en el título como en el autor.

Ejemplos:

- "row" → "J. K. Rowling"
 - "clean" → "Clean Code"
-

Casos de uso

Notificación de venta

Cuando un usuario compra un libro:

1. La compra se registra correctamente.
2. El libro cambia a estado **SOLD**.
3. Se registra la fecha de venta.
4. El vendedor recibe un email notificando la compra.

La implementación del envío puede realizarse:

- de forma síncrona;
 - mediante eventos;
 - mediante colas de procesamiento.
-

Sugerencia de bajada de precio

Una vez por semana se ejecutará una tarea programada.

Comportamiento

La tarea deberá localizar todos los libros que:

- tengan estado **PUBLISHED**;
- lleven publicados más de 7 días.

Por cada libro encontrado se enviará un email al propietario sugiriendo una revisión o bajada de precio para aumentar las posibilidades de venta.

Testing

Se deberán implementar exclusivamente tests E2E.

Como mínimo deberán existir pruebas para los siguientes endpoints:

POST /books

Escenarios recomendados:

- Creación correcta.
- Usuario no autenticado.
- Datos inválidos.

POST /books/:id/buy

Escenarios recomendados:

- Compra correcta.
- Libro inexistente.
- Libro ya vendido.
- Compra de un libro propio.

GET /books

Escenarios recomendados:

- Obtención paginada de resultados.
- Búsqueda parcial por título.
- Búsqueda parcial por autor.
- Exclusión de libros vendidos.

Criterios de aceptación

La práctica se considerará completada cuando:

- Todos los endpoints funcionen correctamente.
- Se respeten todas las reglas de negocio.
- La autenticación se implemente mediante JWT.

- PostgreSQL sea utilizada como base de datos.
- Los endpoints privados estén protegidos mediante middleware.
- Existan tests E2E para los endpoints requeridos.
- La notificación vía correo electrónico debe estar implementada, pero no es necesario que el correo llegue.
- La tarea programada semanal funcione según lo especificado.